Malware Analysis

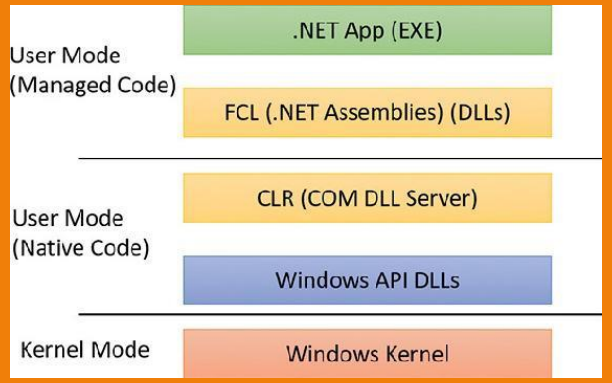
Windows Internals

COMPILED BY: DHARMESH DAVE | ASST. PROF. | NATIONAL FORENSIC SCIENCES UNIVERSITY

Basic Terminology

- **Windows API:**
- The Windows application programming interface (API) is the user-mode system programming interface to the Windows OS family.
- **Windows API functions:**
- These are documented, callable subroutines in the Windows API. Examples include *CreateProcess*, *CreateFile*, and *GetMessage*.
- **Native system services (or system calls):**
- These are the undocumented, underlying services in the OS that are callable from user mode. For example, *NtCreateUserProcess* is the internal system service the Windows *CreateProcess* function calls to create a new process

Basic Terminology



- **Kernel support functions (or routines):**
- These are the subroutines inside the Windows OS that can be called only from kernel mode.
- **Windows services:**
- These are processes started by the Windows service control manager. For example, the Task Scheduler service runs in a user-mode process that supports the *schtasks* command.
- **Dynamic link libraries (DLLs):**
- These are callable subroutines linked together as a binary file that can be dynamically loaded by applications that use the subroutines.

Window Processes

- A **process** is a program being executed by Windows. Each process manages its own resources, such as open handles and memory. A process contains one or more threads that are executed by the CPU.
- Windows uses processes as containers to manage resources and keep separate programs from interfering with each other. There are usually at least 20 to 30 processes running on a Windows system at any one time, all sharing the same resources, including the CPU, file system, memory, and hardware.
- Although programs and processes appear similar on the surface, they are fundamentally different.
- A program is a static sequence of instructions, whereas a process is a container for a set of resources used when executing the instance of the program.

Windows Processes

- **At the highest level of abstraction, a Windows process comprises the following:**
- **A private virtual address space:** This is a set of virtual memory addresses that the process can use.
- **An executable program:** This defines initial code and data and is mapped into the process's virtual address space.
- **A list of open handles:** These map to various system resources such as semaphores, synchronization objects, and files that are accessible to all threads in the process.
- **A security context:** This is an access token that identifies the user, security groups, privileges, attributes, claims, capabilities, User Account Control (UAC) virtualization state, session, and limited user account state associated with the process
- **A process ID:** This is a unique identifier, which is internally part of an identifier called a client ID.

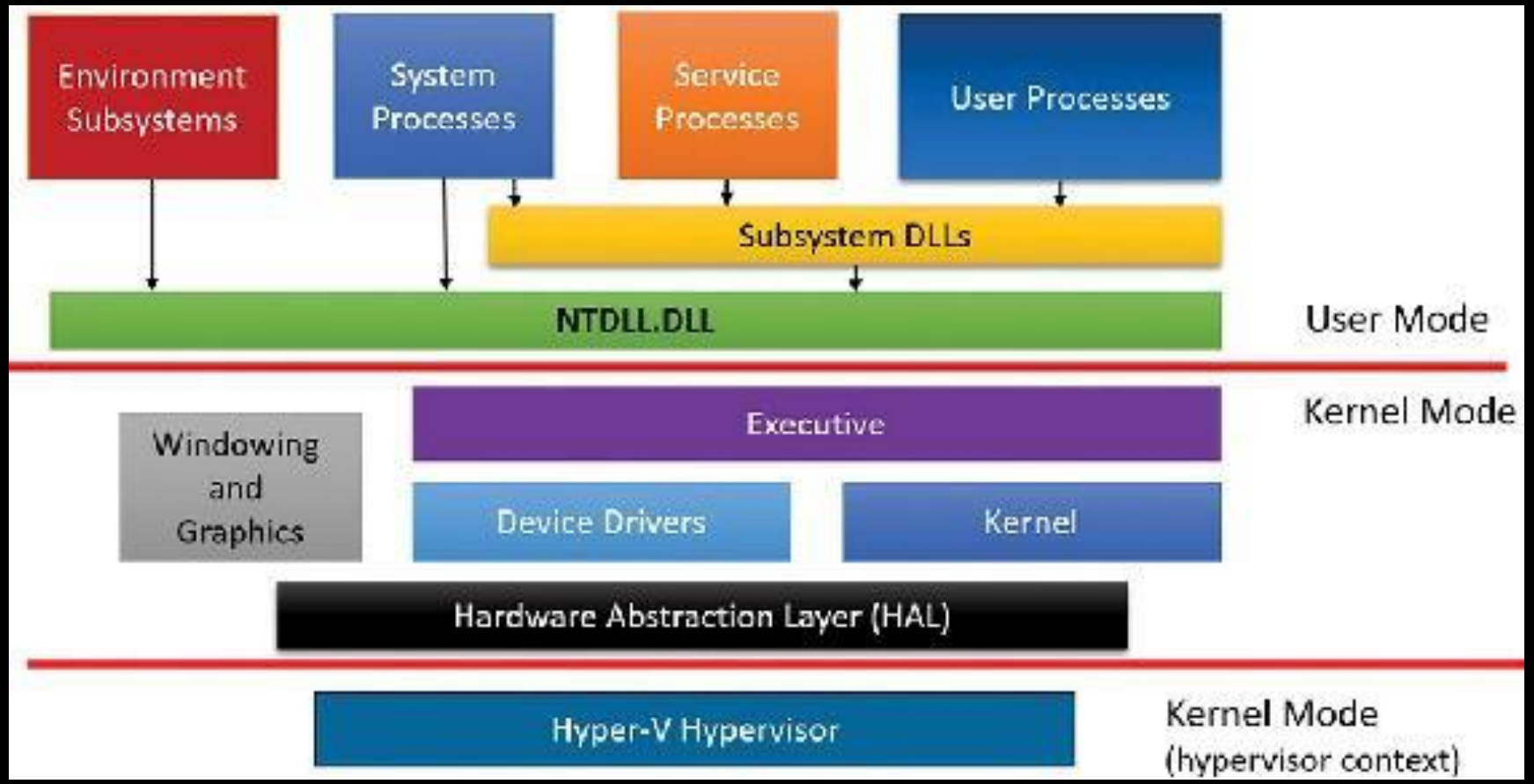
Windows Threads

- A thread is an entity within a process that Windows schedules for execution. Without it, the process's program can't run.
- **A thread includes the following essential components:**
- The contents of a set of CPU registers representing the state of the processor
- Two stacks—one for the thread to use while executing in kernel mode and one for executing in user mode
- A private storage area called thread-local storage (TLS) for use by subsystems, run-time libraries, and DLLs.
- A unique identifier called a thread ID.

Windows Architecture

- In most multiuser operating systems, applications are separated from the OS itself. The OS kernel code runs in a privileged processor mode (referred to as kernel mode), with access to system data and to the hardware.
- Application code runs in a non-privileged processor mode (called user mode), with a limited set of interfaces available, limited access to system data, and no direct access to hardware.
- When a user-mode program calls a system service, the processor executes a special instruction that switches the calling thread to kernel mode.
- When the system service completes, the OS switches the thread context back to user mode and allows the caller to continue.

Windows Architecture



Simplified Windows Architecture

Windows Architecture

- **User processes:** These processes can be one of the following types: Windows 32-bit or 64-bit
- **Service processes:** These are processes that host Windows services, such as the Task Scheduler and Print Spooler services. Services generally have the requirement that they run independently of user logons.
- **System processes:** These are fixed, or hardwired, processes, such as the logon process and the Session Manager, that are not Windows services. That is, they are not started by the Service Control Manager.
- **Environment subsystem server processes:** These implement part of the support for the OS environment, or personality, presented to the user and programmer.

Windows Architecture

- **The Subsystem DLLs:** User applications don't call the native Windows OS services directly. Rather, they go through one or more subsystem dynamic-link libraries (DLLs).
- **Executive:** The Windows executive contains the base OS services, such as memory management, process and thread management, security, I/O, networking, and inter-process communication.
- **The Windows kernel:** This consists of low-level OS functions, such as thread scheduling, interrupt and exception dispatching, and multiprocessor synchronization.
- **Device drivers:** This includes both hardware device drivers, which translate user I/O function calls into specific hardware device I/O requests, and non-hardware device drivers, such as file system and network drivers.

Windows Architecture

- **The Hardware Abstraction Layer (HAL):** This is a layer of code that isolates the kernel, the device drivers, and the rest of the Windows executive from platform-specific hardware differences
- **The windowing and graphics system** This implements the graphical user interface (GUI) functions
- **Hypervisor:** To provide such virtualization services, almost all modern solutions employ the use of a hypervisor, which is a specialized and highly privileged component that allows for the virtualization and isolation of all resources on the machine, from virtual to physical memory, to device interrupts, and even to PCI and USB devices.

Environment Subsystems and Subsystem DLL

- The role of an **environment subsystem** is to expose some subset of the base Windows executive system services to application programs.
- Each executable image (.exe) is bound to one and only one subsystem. When an image is run, the process creation code examines the subsystem type code in the image header so that it can notify the proper subsystem of the new process.
- User applications don't call Windows system services directly. Instead, they go through one or more subsystem DLLs. These libraries export the documented interface that the programs linked to that subsystem can call.
- For example, the **Windows subsystem DLLs (such as Kernel32.dll, Advapi32.dll, User32.dll, and Gdi32.dll)** implement the Windows API functions.

Environment Subsystems and Subsystem DLL

- When an application calls a function in a subsystem DLL, one of **three things can occur**:
- 1) The function is entirely implemented in user mode inside the subsystem DLL. In other words, no message is sent to the environment subsystem process, and no Windows executive system services are called. The function is performed in user mode, and the results are returned to the caller.
- Example like ***GetCurrentProcess*** always return -1 value refer to current process and does not change in running process, avoid need to call into the kernel.

Environment Subsystems and Subsystem DLL

- When an application calls a function in a subsystem DLL, one of three things can occur:
- 2) The function requires one or more calls to the Windows executive.
- For example, the Windows ***ReadFile*** and ***WriteFile*** functions involve calling the underlying internal Windows I/O system services ***NtReadFile*** and ***NtWriteFile***, respectively.
- 3) The function requires some work to be done in the environment subsystem process.
- In this case, a client/server request is made to the environment subsystem via an ALPC message sent to the subsystem to perform some operation. The subsystem DLL then waits for a reply before returning to the caller.

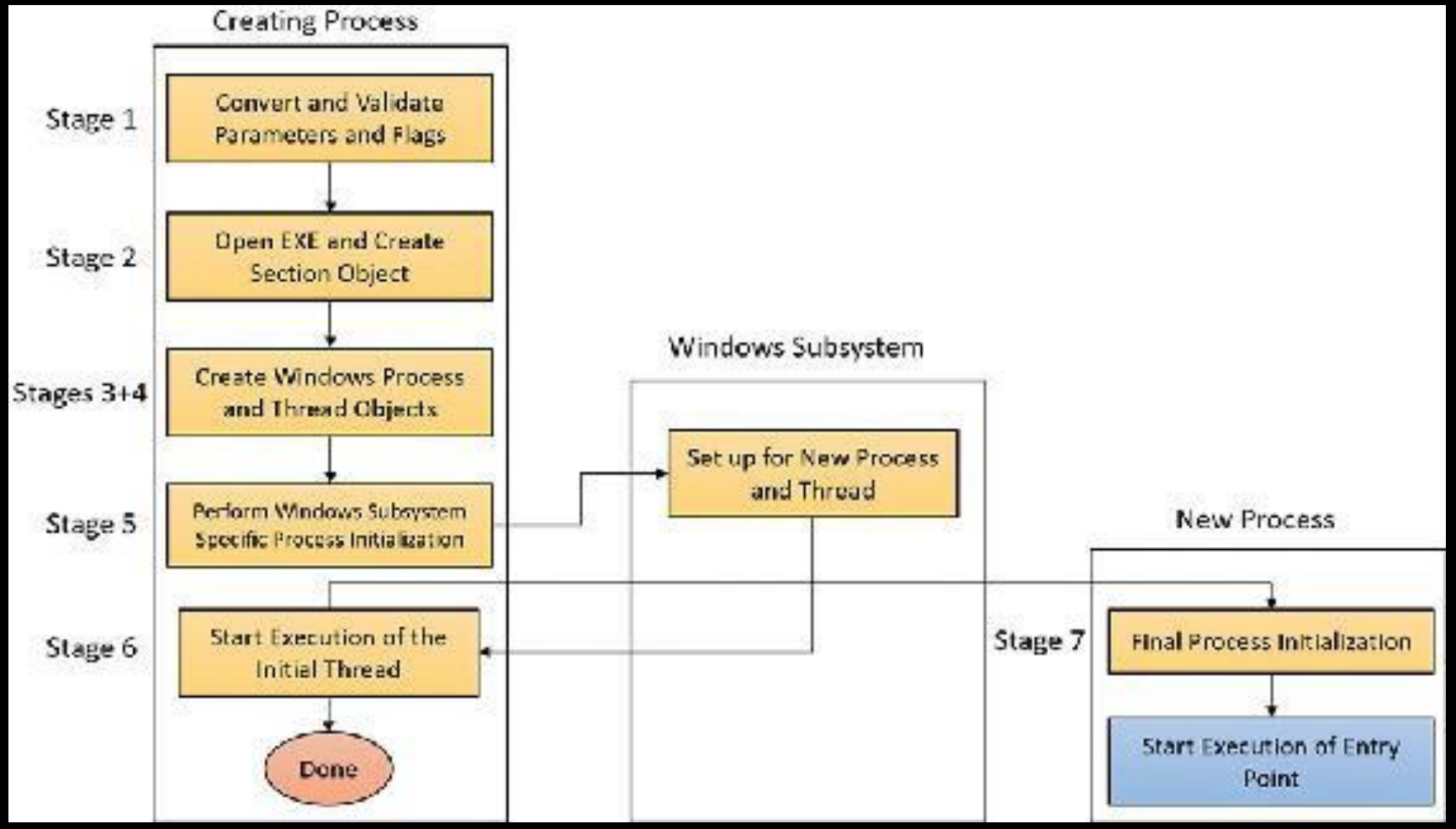
Creating a process

- The Windows API provides several functions for creating processes. The simplest is ***CreateProcess***, which attempts to create a process with the same access token as the creating process.
- If a different token is required, *CreateProcessAsUser* can be used, which accepts an extra argument (the first)—a handle to a token object that was already somehow obtained.
- CreateProcessWithTokenW is similar to CreateProcessAsUser, but the two differ in the privileges required for the caller.
- CreateProcessWithLogonW is a handy shortcut to log on with a given user's credentials and create a process with the obtained token in one stroke.

Flow of Create Process

- Creating a Windows process consists of several stages carried out in three parts of the operating system: the Windows client-side library Kernel32.dll the Windows executive, and the Windows subsystem process.
- ***CreateProcess*** steps:
  1. Validate parameters; convert Windows subsystem flags and options to their native counterparts; parse, validate, and convert the attribute list to its native counterpart.
  2. Open the image file (.exe) to be executed inside the process.
  3. Create the Windows executive process object.
  4. Create the initial thread (stack, context, and Windows executive thread object).
  5. Perform post-creation, Windows subsystem–specific process initialization.
  6. Start execution of the initial thread.
  7. In the context of the new process and thread, complete the initialization of the address space (for example, load required DLLs) and begin execution of the program's entry point.

Flow of Create Process



The main stages of process creation

Flow of Create Process

- BOOL **CreateProcessA** **(**
- [in, optional] LPCSTR lpApplicationName,
- [in, out, optional] LPSTR lpCommandLine,
- [in, optional] LPSECURITY_ATTRIBUTES lpProcessAttributes,
- [in, optional] LPSECURITY_ATTRIBUTES lpThreadAttributes,
- [in] BOOL bInheritHandles,
- [in] DWORD **dwCreationFlags**,
- [in, optional] LPVOID lpEnvironment,
- [in, optional] LPCSTR lpCurrentDirectory,
- [in] LPSTARTUPINFOA lpStartupInfo,
- [out] LPPROCESS_INFORMATION lpProcessInformation**);**

https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessa

Terminating a Process

- A process can exit gracefully by calling the ***ExitProcess*** function. the process startup code for the first thread calls ExitProcess on the process's behalf when the thread returns from its main function.
- ***ExitProcess*** can be called only by the process itself asking to exit. An ungraceful termination of a process is possible using the ***TerminateProcess*** function, which can be called from outside the process. (For example, Process Explorer and Task Manager use it when so requested.)

Protected Process

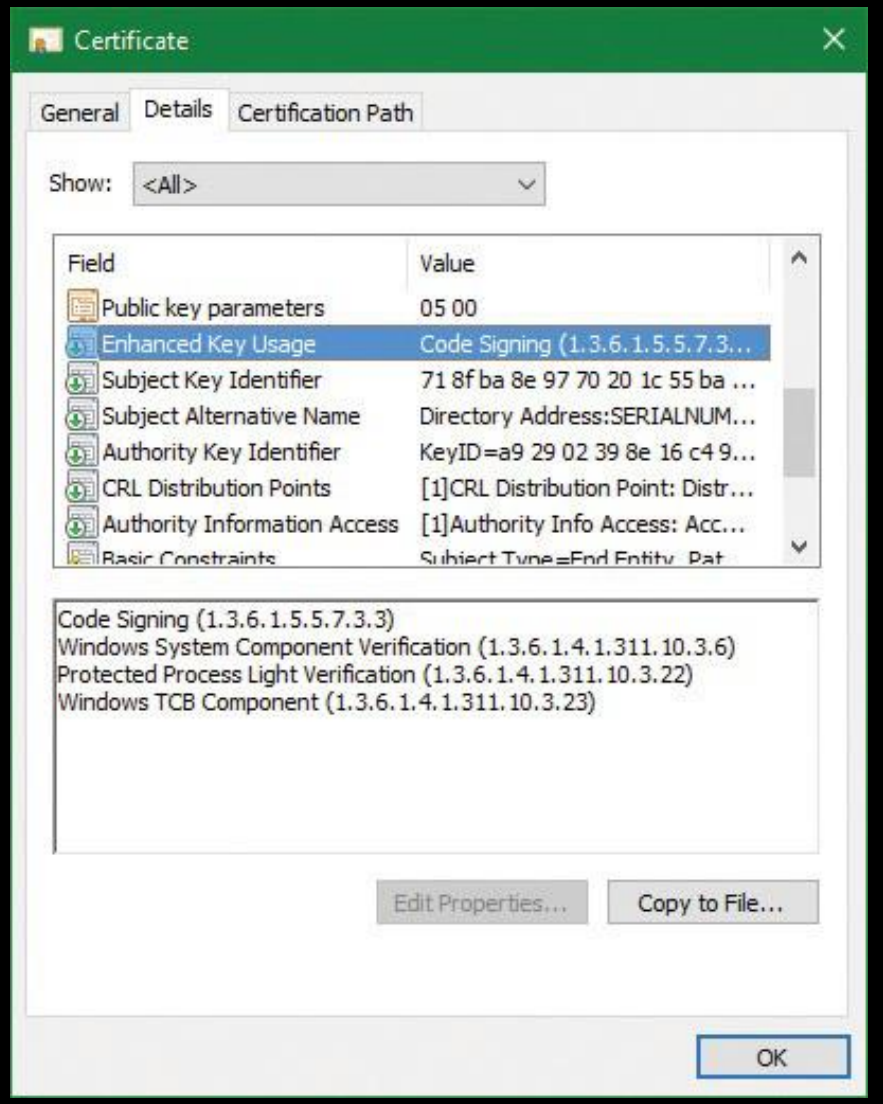
- In the Windows security model, any process running with a token containing the debug privilege (such as an administrator's account) can request any access right that it desires to any other process running on the machine.
- For example, it can read and write arbitrary process memory, inject code, suspend and resume threads, and query information on other processes.
- Tools such as Process Explorer and Task Manager need and request these access rights to provide their functionality to users.

Protected Process

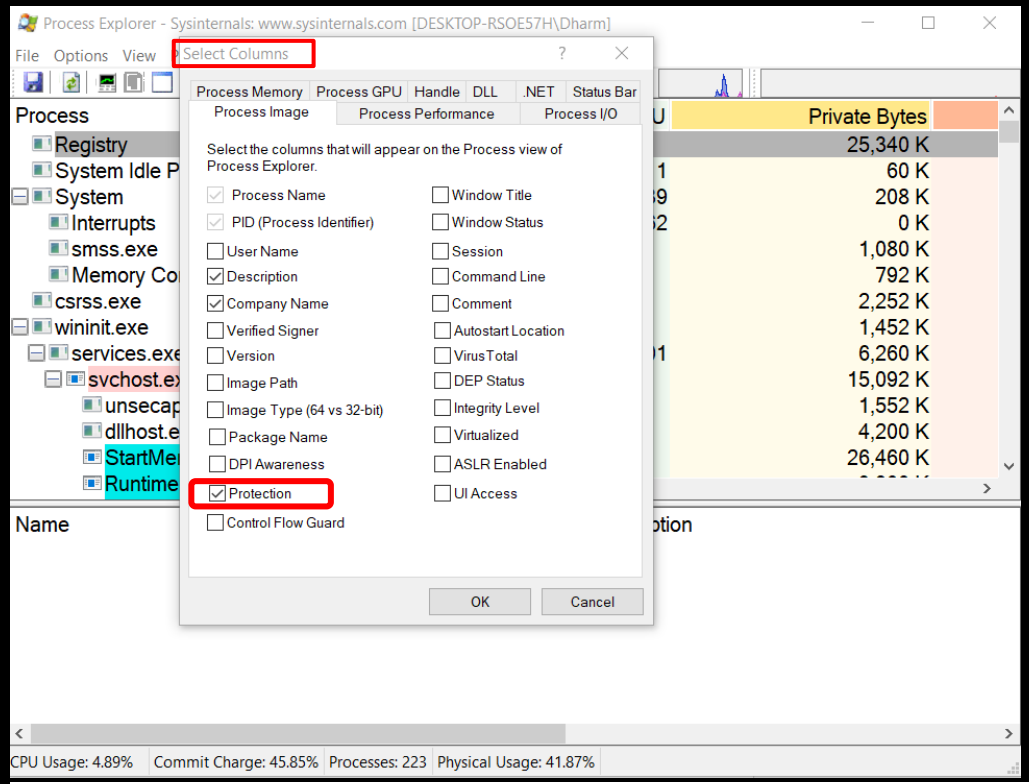
- This logical behavior *(which helps ensure that administrators will always have full control of the running code on the system)* clashes with the system behavior for digital rights management (DRM) requirements imposed by the media industry on computer operating systems that need to support playback of advanced, high-quality digital content such as Blu-ray media.
- To support reliable and protected playback of such content, Windows Vista and Windows Server 2008 introduced protected processes.
- Protected processes can be created by any application. However, the operating system will allow a process to be protected only if the image file has been digitally signed with a special Windows Media Certificate.

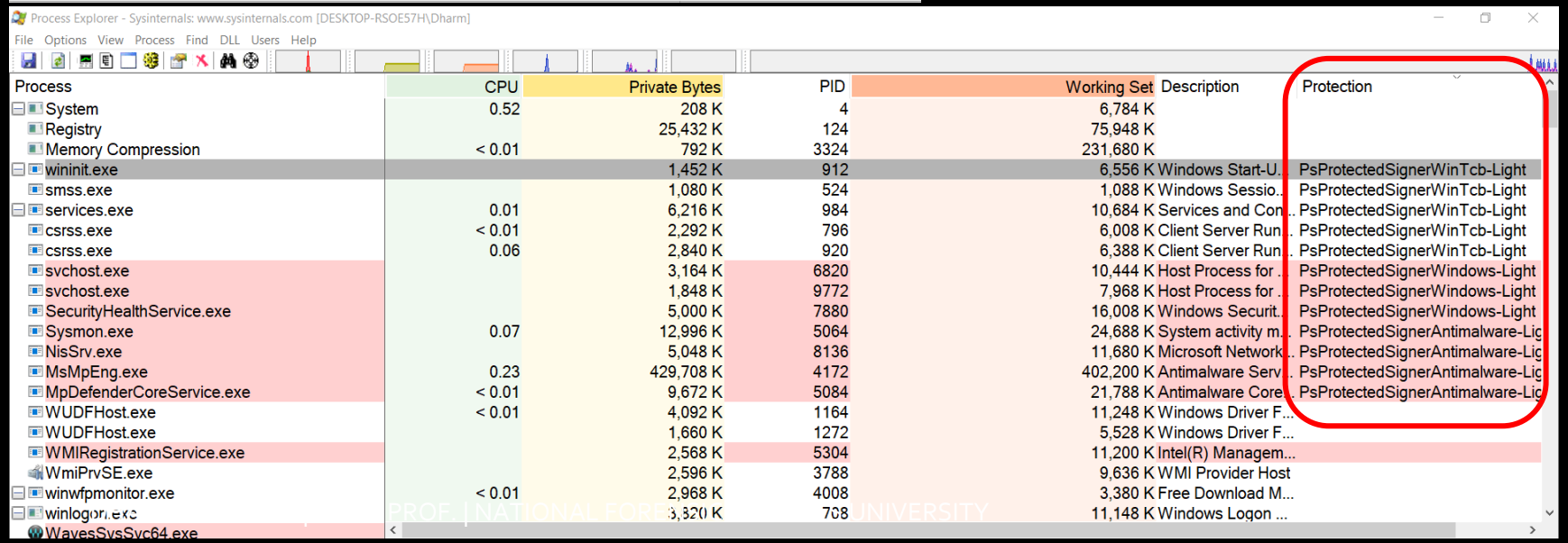
Protected Process

- The Protected Media Path (PMP) in Windows makes use of protected processes to provide protection for high-value media, and developers of applications such as DVD players can make use of protected processes by using the Media Foundation (MF) API.



Protected Process





Viewing protected processes in Process Explorer

Creating Threads

- Processes are the container for execution, but threads are what the Windows OS executes. **Threads** are independent sequences of instructions that are executed by the CPU without waiting for other threads.
- A process contains one or more threads, which execute part of the code within a process. Threads within a process all share the same memory space, but each has its own processor registers and stack.
- When a thread changes the value of a register in a CPU, it does not affect any other threads. Before an OS switches between threads, all values in the CPU are saved in a structure called the thread context.

Creating Threads

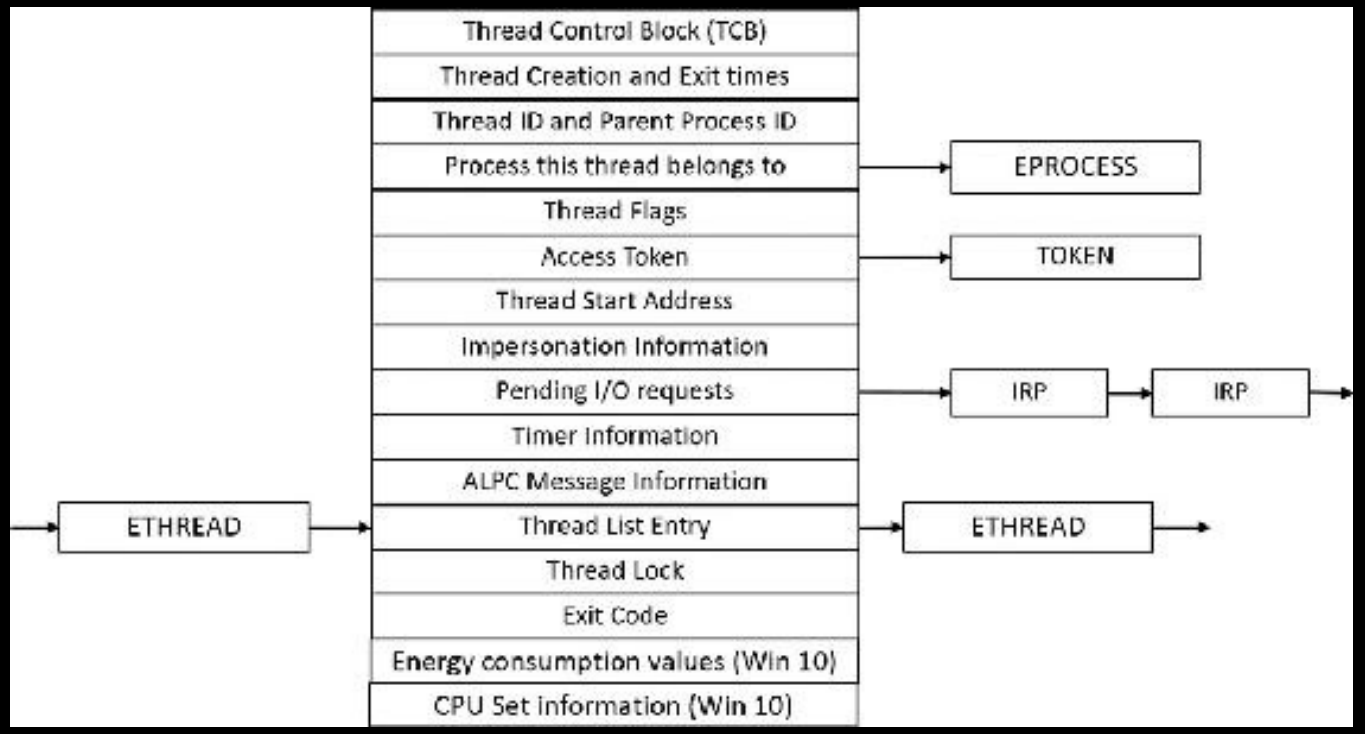
- The simplest creation function in user mode is ***CreateThread***. This function creates a thread in the current process, accepting the **following arguments**:
- **An optional security attributes structure** This specifies the security descriptor to attach to the newly created thread. It also specifies whether the thread handle is to be created as inheritable.
- **An optional stack size** If zero is specified, a default is taken from the executable's header. This always applies to the first thread in a user-mode process.
- **A function pointer** This serves as the entry point for the new thread's execution
- **An optional argument** This is to pass to the thread's function.
- **Optional flags** One controls whether the thread starts suspended

Creating Threads

- An extended thread creation function is ***CreateRemoteThread***. This function accepts an extra argument (the first), which is a handle to a target process where the thread is to be created. You can use this function to inject a thread into another process.
- CreateRemoteThreadEx eventually calls ***NtCreateThreadEx*** in Ntdll.dll. This makes the usual transition to kernel mode, where execution continues in the executive function NtCreateThreadEx.

Thread Internals

- At the operating-system (OS) level, a Windows thread is represented by an executive thread object. The executive thread object encapsulates an ETHREAD structure, which in turn contains a KTHREAD (Kernel Thread) structure as its first member.



Thread Internals

- HANDLE **CreateThread** (
- [in, optional] LPSECURITY_ATTRIBUTES lpThreadAttributes,
- [in] SIZE_T dwStackSize,
- [in] LPTHREAD_START_ROUTINE **lpStartAddress,**
- [in, optional] __drv_aliasesMem LPVOID lpParameter,
- [in] DWORD dwCreationFlags,
- [out, optional] LPDWORD lpThreadId);

https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createremotethread

Birth of Thread *(CreateThread)*

1. The function converts the Windows API parameters to native flags and builds a native structure describing object parameters
2. It builds an attribute list with two entries: client ID and TEB address.
3. It determines whether the thread is created in the calling process or another process indicated by the handle passed in.
4. It calls NtCreateThreadEx (in Ntdll) to make the transition to the executive in kernel mode and continues inside a function with the same name and arguments.
5. NtCreateThreadEx (inside the executive) creates and initializes the user-mode thread context
6. CreateRemoteThreadEx allocates an activation context for the thread used by side-by-side assembly support.
7. Unless the caller created the thread with the CREATE_SUSPENDED flag set, the thread is now resumed so that it can be scheduled for execution.
8. The thread handle and the thread ID are returned to the caller.

References

- Windows Internals Seventh Edition Part 1: System architecture, processes, threads, memory management, and more by Pavel Yosifovich, Alex Ionescu, Mark E. Russinovich, and David A. Solomon.
- Practical Malware Analysis by Michael Sikorski